

MEDS Lab - Module 4

Section 9 Report: Flip-Flops, Registers & Reset

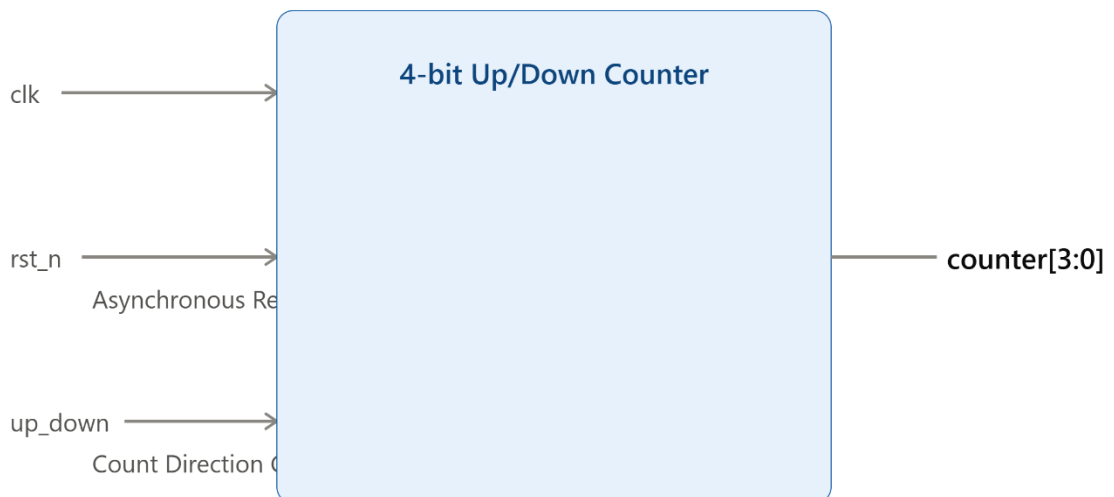
Name: Muhammad Kaleem Faryad

University: UET Lahore

Task 10: 4-bit Up-Down Counter with Asynchronous Reset

1. EDA Playground Link: <https://edaplayground.com/x/izGG>

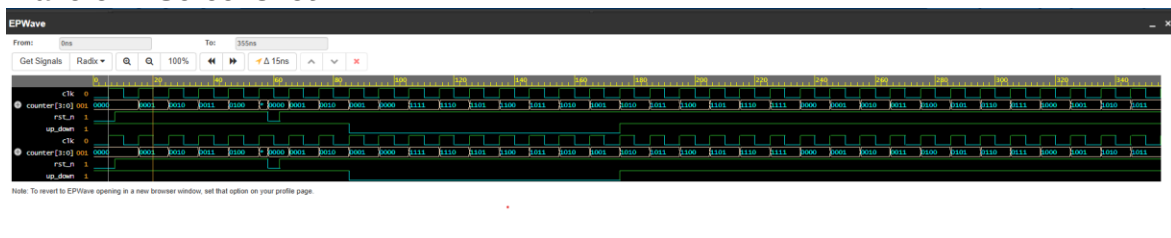
2. Block Diagram:



3. Simulation Output Screenshot:

```
Log Share
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 22 08:10 2026
Time=0 clk=0 rst_n=0 up_down=1 counter=0
Time=5 clk=1 rst_n=0 up_down=1 counter=0
Time=7 clk=1 rst_n=1 up_down=1 counter=0
Time=10 clk=0 rst_n=1 up_down=1 counter=0
Time=15 clk=1 rst_n=1 up_down=1 counter=1
Time=20 clk=0 rst_n=1 up_down=1 counter=1
Time=25 clk=1 rst_n=1 up_down=1 counter=2
Time=30 clk=0 rst_n=1 up_down=1 counter=2
Time=35 clk=1 rst_n=1 up_down=1 counter=3
Time=40 clk=0 rst_n=1 up_down=1 counter=3
Time=45 clk=1 rst_n=1 up_down=1 counter=4
Time=50 clk=0 rst_n=1 up_down=1 counter=4
Time=55 clk=1 rst_n=1 up_down=1 counter=5
Time=58 clk=1 rst_n=0 up_down=1 counter=0
Time=60 clk=0 rst_n=0 up_down=1 counter=0
Time=62 clk=0 rst_n=1 up_down=1 counter=0
Time=65 clk=1 rst_n=1 up_down=1 counter=1
Time=70 clk=0 rst_n=1 up_down=1 counter=1
Time=75 clk=1 rst_n=1 up_down=1 counter=2
Time=80 clk=0 rst_n=1 up_down=1 counter=2
Time=85 clk=1 rst_n=1 up_down=0 counter=1
Time=90 clk=0 rst_n=1 up_down=0 counter=1
Time=95 clk=1 rst_n=1 up_down=0 counter=0
Time=100 clk=0 rst_n=1 up_down=0 counter=0
Time=105 clk=1 rst_n=1 up_down=0 counter=15
Time=110 clk=0 rst_n=1 up_down=0 counter=15
Time=115 clk=1 rst_n=1 up_down=0 counter=14
Time=120 clk=0 rst_n=1 up_down=0 counter=14
Time=125 clk=1 rst_n=1 up_down=0 counter=13
Time=130 clk=0 rst_n=1 up_down=0 counter=13
Time=135 clk=1 rst_n=1 up_down=0 counter=12
Time=140 clk=0 rst_n=1 up_down=0 counter=12
Time=145 clk=1 rst_n=1 up_down=0 counter=11
Time=150 clk=0 rst_n=1 up_down=0 counter=11
Time=155 clk=1 rst_n=1 up_down=0 counter=10
Time=160 clk=0 rst_n=1 up_down=0 counter=10
Time=165 clk=1 rst_n=1 up_down=0 counter=9
Time=170 clk=0 rst_n=1 up_down=0 counter=9
Time=175 clk=1 rst_n=1 up_down=1 counter=10
Time=180 clk=0 rst_n=1 up_down=1 counter=10
Time=185 clk=1 rst_n=1 up_down=1 counter=11
Time=190 clk=0 rst_n=1 up_down=1 counter=11
Time=195 clk=1 rst_n=1 up_down=1 counter=12
Time=200 clk=0 rst_n=1 up_down=1 counter=12
```

4. Waveform Screenshot:



5. **Explanation:** A 4-bit up-down counter with an active-low asynchronous reset was implemented using an `always_ff` block. The counter stores its current value in a 4-bit register and updates only on the positive edge of the clock. The `up_down` input controls the counting direction. When `up_down = 1`, the counter increments by one at every rising clock edge. When `up_down = 0`, it decrements by one. Since the counter is 4 bits wide, it automatically wraps around from 15 to 0 during counting up and from 0 to 15 during counting down.

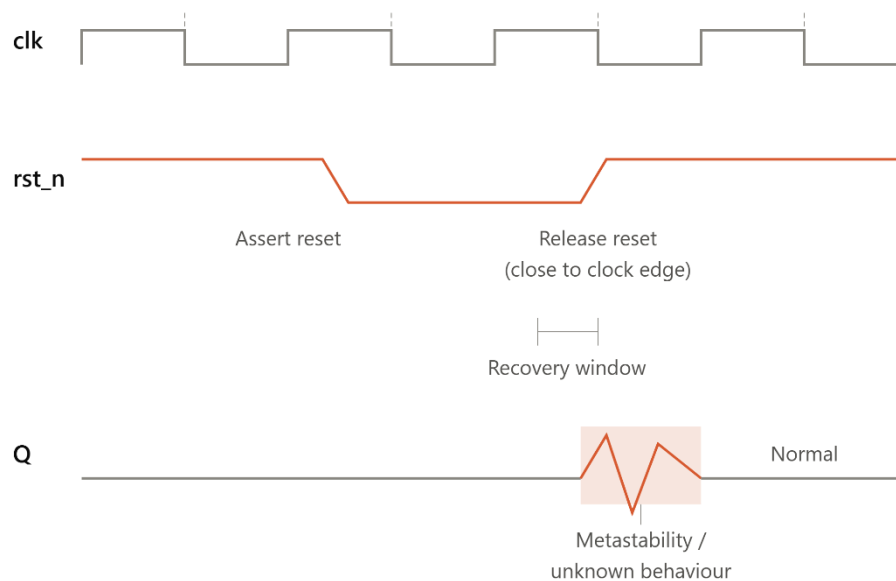
An active-low asynchronous reset (`rst_n`) was included in the sensitivity list (`posedge clk or negedge rst_n`). Whenever `rst_n` becomes 0, the counter immediately resets to 0000 regardless of the clock. This behavior is different from a

synchronous reset because it does not wait for the next clock edge. When `rst_n` returns to 1, the counter resumes normal operation from zero on subsequent clock edges.

The testbench generated a periodic clock, applied an asynchronous reset at the beginning of the simulation, counted upward for several clock cycles, asserted reset in the middle of the counting sequence to verify that the counter cleared immediately, and then released reset to continue operation. The direction was later changed to count downward, demonstrating the wraparound from 0 to 15. Finally, the direction was switched back to count upward to verify the wraparound from 15 to 0. The simulation results confirmed that the counter satisfied all assignment requirements, including immediate asynchronous reset, correct up and down counting, and proper wraparound behavior.

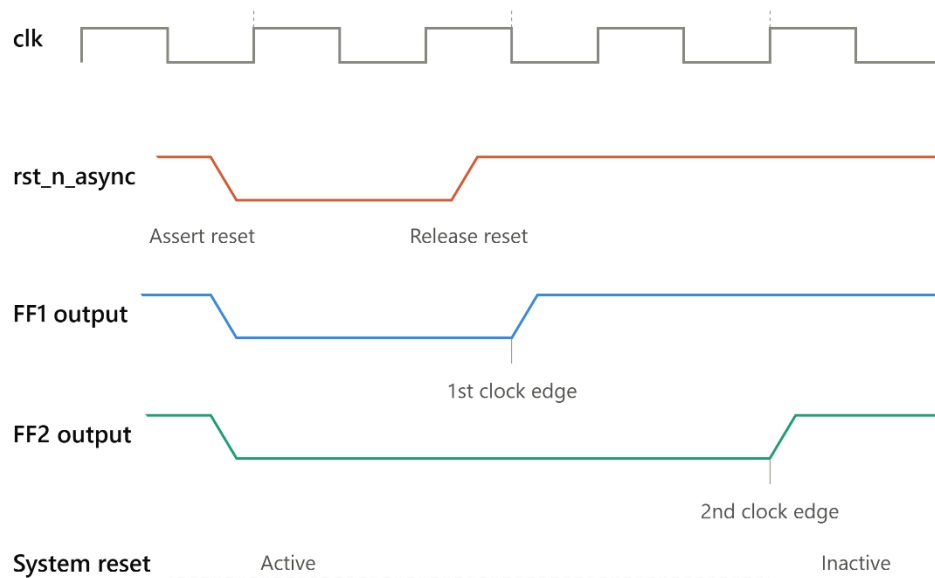
Task 11: Reset Synchroniser

1. EDA Playground Link: <https://edaplayground.com/x/DHja>
2. Timing Diagram Showing the Problem (Unsynchronized Asynchronous Reset):



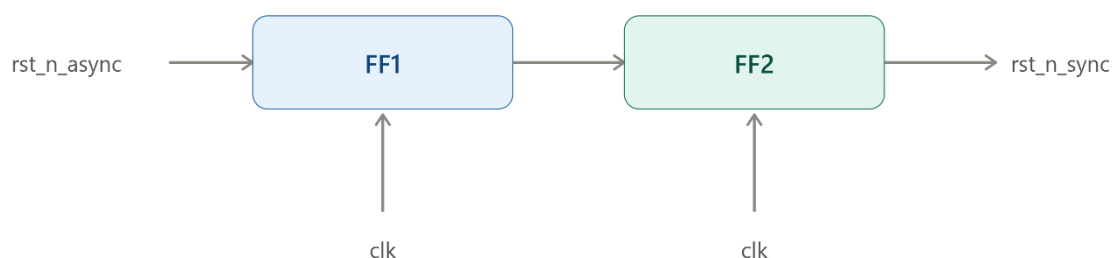
Explanation: When `rst_n_async` is asserted (changes from 1 to 0), the flip-flop is reset immediately, which is safe because the output is simply forced to zero. However, when the reset is released (changes from 0 to 1), the transition occurs very close to a rising clock edge. This violates the flip-flop's recovery-time requirement. As a result, the flip-flop may enter a metastable state where its output becomes temporarily unpredictable before settling to either logic 0 or logic 1. Different flip-flops in the same design may recover at different times, causing incorrect system behaviour. Therefore, directly distributing an asynchronous reset is unsafe unless its deassertion is synchronized.

3. Timing Diagram Showing the Reset Synchronizer:



Explanation: The asynchronous reset (`rst_n_async`) is connected directly to the asynchronous reset inputs of both flip-flops. Therefore, when the reset is asserted ($1 \rightarrow 0$), both flip-flops reset immediately without waiting for a clock edge. During reset deassertion ($0 \rightarrow 1$), the reset signal is not passed directly to the rest of the circuit. Instead, it propagates through two cascaded flip-flops. On the first rising edge of the clock, the first flip-flop captures the released reset while the second flip-flop still holds the reset active. On the second rising edge, the second flip-flop also captures the released reset, causing the synchronized reset (`rst_n_sync`) to become high. As a result, the reset is released only on a clock edge and after two clock cycles, ensuring that all downstream flip-flops leave reset simultaneously. This eliminates recovery-time violations and greatly reduces the possibility of metastability in the design.

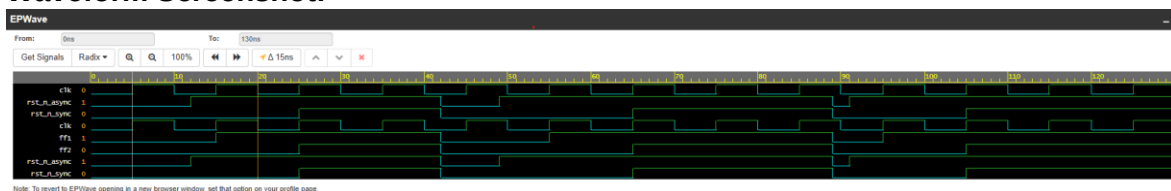
4. Block Diagram:



5. Simulation Output Screenshot:

```
Log Share
Compiler version X-2025.06-SP1-Full164; Kunttime version X-2025.06-SP1-Full164; JUL 22 13:39 2026
Time=0 clk=0 rst_n_async=0 rst_n_sync=0
Time=5 clk=1 rst_n_async=0 rst_n_sync=0
Time=10 clk=0 rst_n_async=0 rst_n_sync=0
Time=12 clk=0 rst_n_async=1 rst_n_sync=0
Time=15 clk=1 rst_n_async=1 rst_n_sync=0
Time=20 clk=0 rst_n_async=1 rst_n_sync=0
Time=25 clk=1 rst_n_async=1 rst_n_sync=1
Time=30 clk=0 rst_n_async=1 rst_n_sync=1
Time=35 clk=1 rst_n_async=1 rst_n_sync=1
Time=40 clk=0 rst_n_async=1 rst_n_sync=1
Time=42 clk=0 rst_n_async=0 rst_n_sync=0
Time=45 clk=1 rst_n_async=0 rst_n_sync=0
Time=49 clk=1 rst_n_async=1 rst_n_sync=0
Time=50 clk=0 rst_n_async=1 rst_n_sync=0
Time=55 clk=1 rst_n_async=1 rst_n_sync=0
Time=60 clk=0 rst_n_async=1 rst_n_sync=0
Time=65 clk=1 rst_n_async=1 rst_n_sync=1
Time=70 clk=0 rst_n_async=1 rst_n_sync=1
Time=75 clk=1 rst_n_async=1 rst_n_sync=1
Time=80 clk=0 rst_n_async=1 rst_n_sync=1
Time=85 clk=1 rst_n_async=1 rst_n_sync=1
Time=89 clk=1 rst_n_async=0 rst_n_sync=0
Time=90 clk=0 rst_n_async=0 rst_n_sync=0
Time=91 clk=0 rst_n_async=1 rst_n_sync=0
Time=95 clk=1 rst_n_async=1 rst_n_sync=0
Time=100 clk=0 rst_n_async=1 rst_n_sync=0
Time=105 clk=1 rst_n_async=1 rst_n_sync=1
Time=110 clk=0 rst_n_async=1 rst_n_sync=1
Time=115 clk=1 rst_n_async=1 rst_n_sync=1
Time=120 clk=0 rst_n_async=1 rst_n_sync=1
Time=125 clk=1 rst_n_async=1 rst_n_sync=1
Time=130 clk=0 rst_n_async=1 rst_n_sync=1
$finish called from file "testbench.sv", line 62.
$finish at simulation time 131
VCS Simulation Report
Time: 131 ns
CPU Time: 0.470 seconds; Data structure size: 0.0Mb
Wed Jul 22 13:39:21 2026
Finding VCD file...
```

6. Waveform Screenshot:



7. **Explanation:** A two-flip-flop reset synchronizer was implemented to safely synchronize the deassertion of an active-low asynchronous reset. The synchronizer consists of two cascaded D flip-flops that share the same clock and asynchronous reset input. When `rst_n_async` is asserted (changes from 1 to 0), both flip-flops are reset immediately because the asynchronous reset is included in the sensitivity list. This allows the entire system to enter the reset state without waiting for a clock edge. When the asynchronous reset is released (changes from 0 to 1), the release is not immediately propagated to the output. Instead, the first flip-flop captures the released reset on the next rising edge of the clock while the second flip-flop remains in the reset state. On the following clock edge, the second flip-flop captures the output of the first

flip-flop, causing the synchronized reset signal (`rst_n_sync`) to become high. Consequently, the reset is released only after two clock cycles and always on a clock edge. This synchronized deassertion prevents recovery-time violations and significantly reduces the possibility of metastability in downstream sequential logic. The testbench verified the design by asserting and deasserting the asynchronous reset at different times relative to the clock, including very close to clock edges. The simulation waveform confirmed that reset assertion occurred immediately, while reset deassertion was delayed by exactly two clock cycles and always aligned with the rising edge of the clock, demonstrating the correct operation of the reset synchronizer.